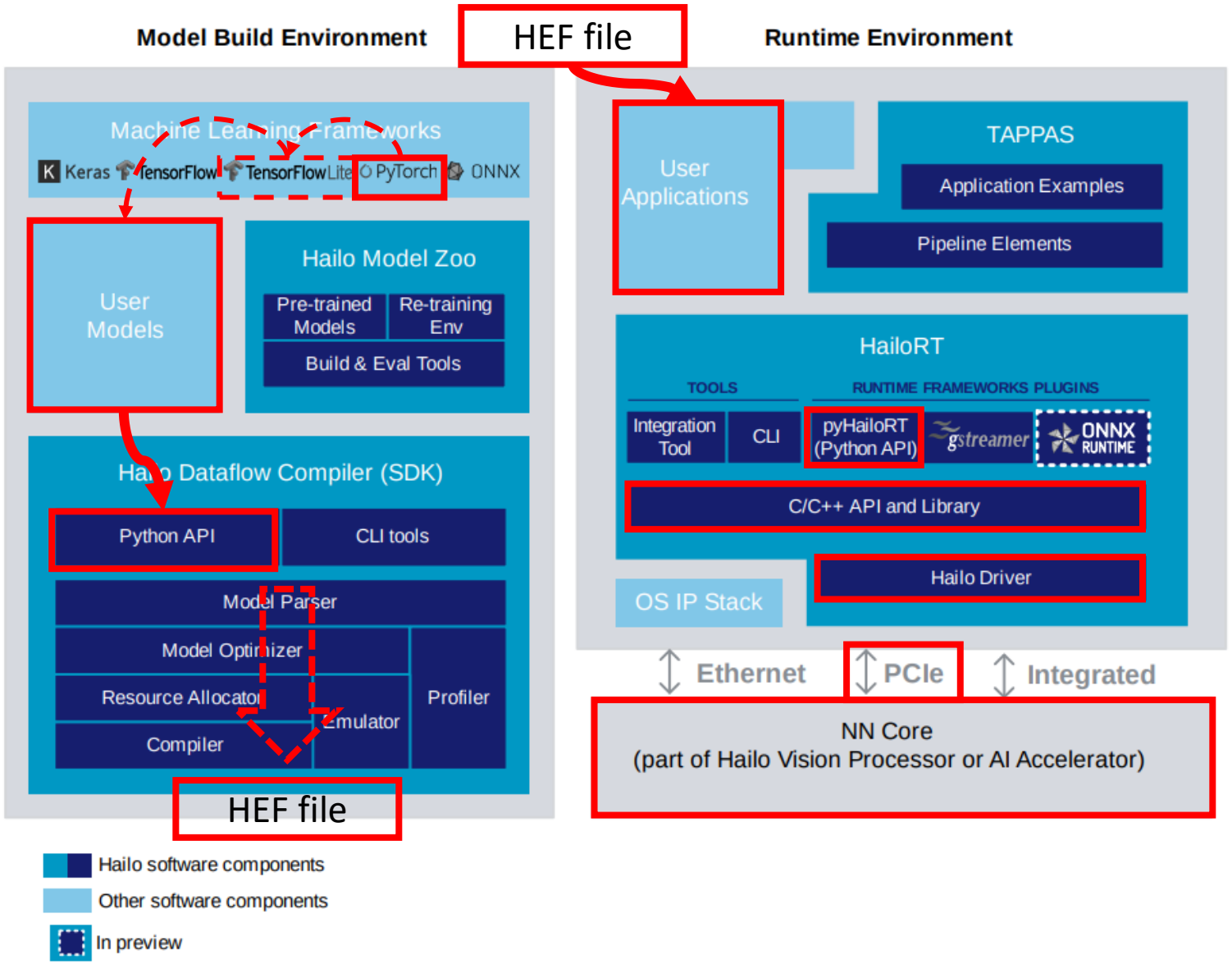
Hailo AI Accelerator with Raspberry Pi 5

For using ultralytics YOLO package

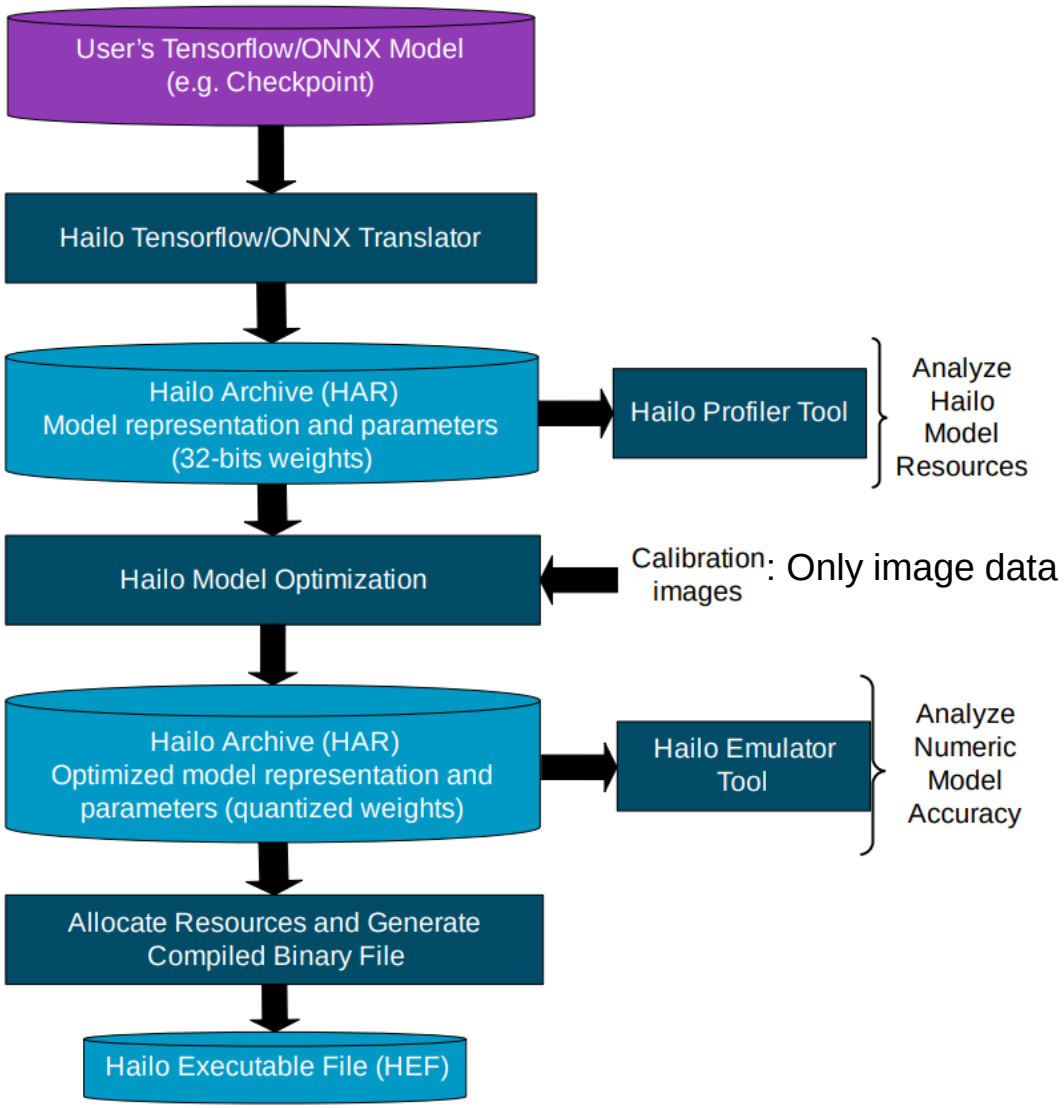
Description of development env.

H.W, O.S., and Hailo S.W. stack.

Hailo S.W. stack

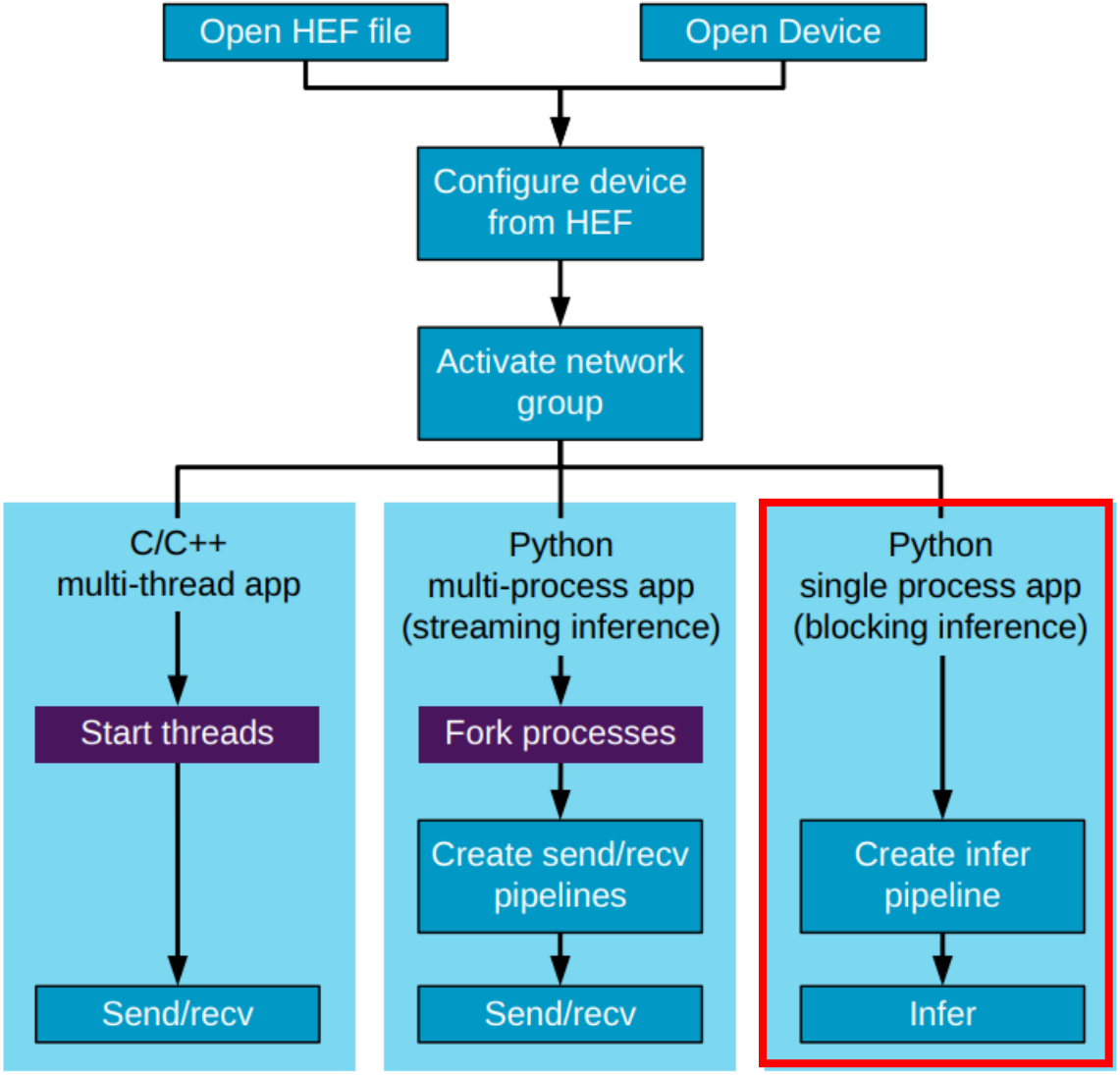


Hailo DataFlow Compiler process



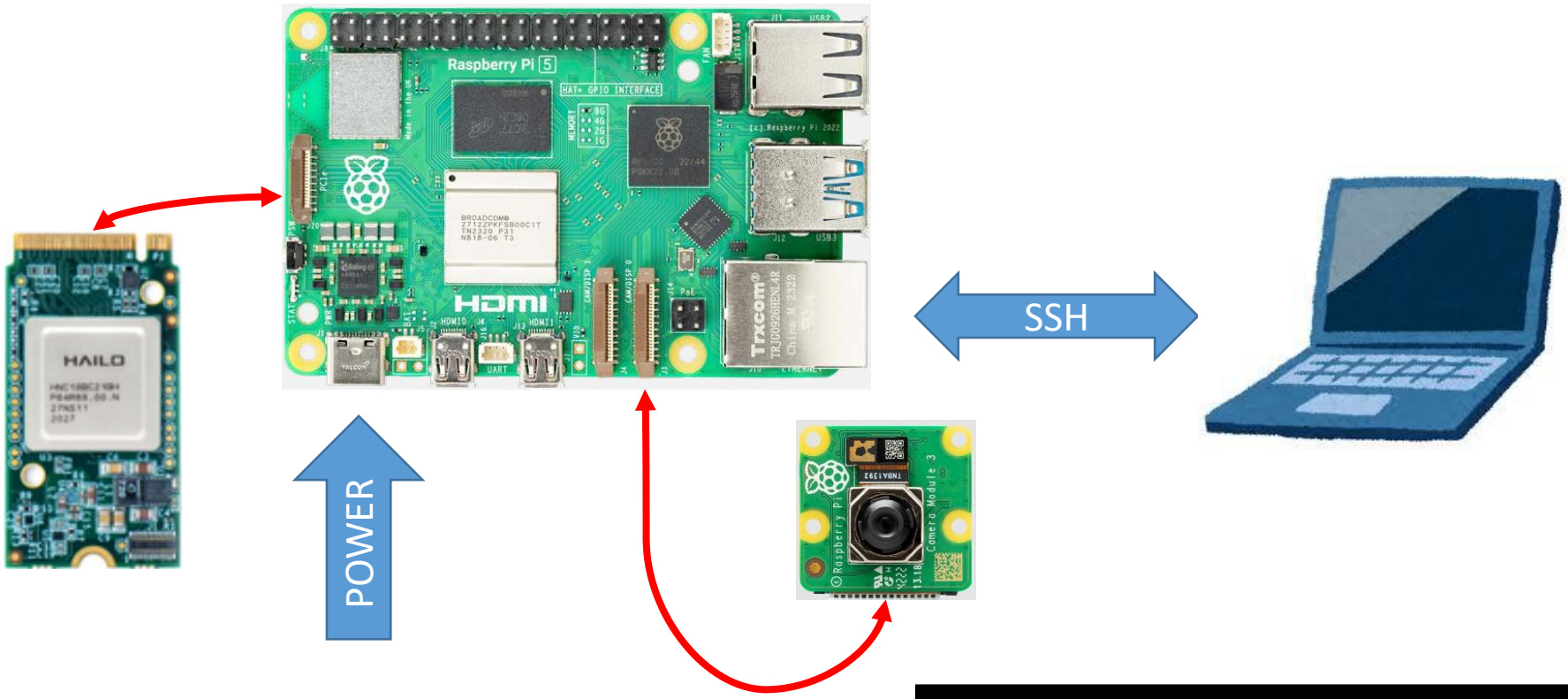
System requirement	
OS	Ubuntu 20.04, 22.04 (WSL2 supported)
CPU	X86_64 arch. (supporting AVX recommended)
RAM	16+ GB, 32+ GB recommended
Python Env.	`Python 3.8/3.9/3.10` `Python pack: disutils, tk, libfuse2, graphviz, libgraphviz-dev`

Hailo RT inference flow



System requirement	
OS	Linux or Windows
RAM	2+ GB, 4+ GB recommended
Acc. board	Hailo-8, 8L
Python	`Python 3.8 - 3.11`

Inference H.W. system



System	
OS	Rasberry Pi OS (Linux, debian)
RAM	8 GB
Acc. Board, Interface	Hailo-8L (M.2 PCIe 2.0) (experimentaly PCIe 3.0)

Dataflow Compiler tutorial

Preparing development environment

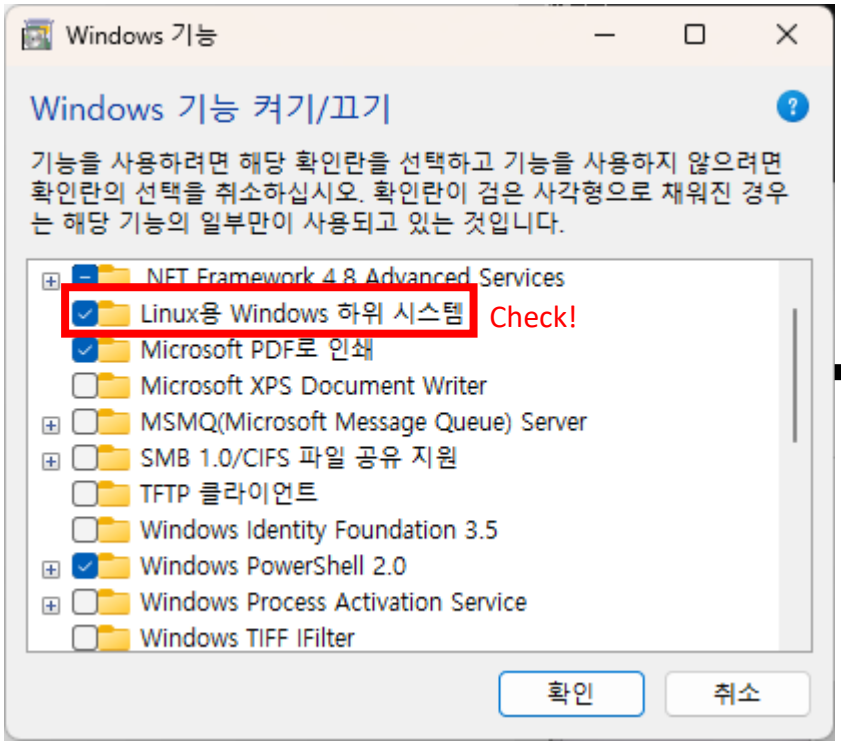
Exporting YOLO model from ultralytics package

Parsing model from Tensorflow Lite model

Model optimization (Post Training Quantization)

Model compiling

Preparing dev. env. (WSL2)



PowerShell (관리자 모드)

```
wsl --install
```

→ Windows reboot

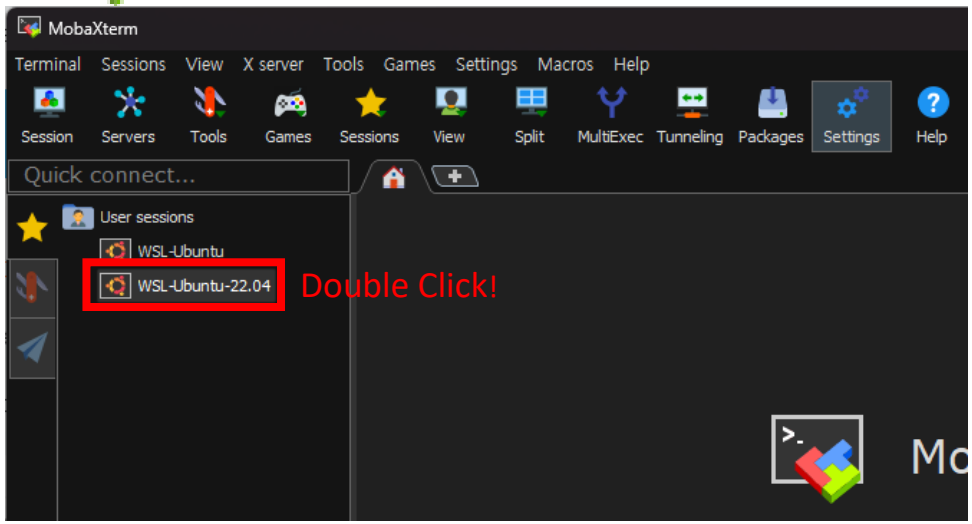
Preparing dev. env. (WSL2)

PowerShell (관리자 모드)

```
wsl --list -online # 설치 가능한 배포판 나열
wsl --install --distribute Ubuntu-22.04 # Ubuntu-22.04 설치
Wsl --distribute Ubuntu-22.04 # Ubuntu-22.04 실행
```

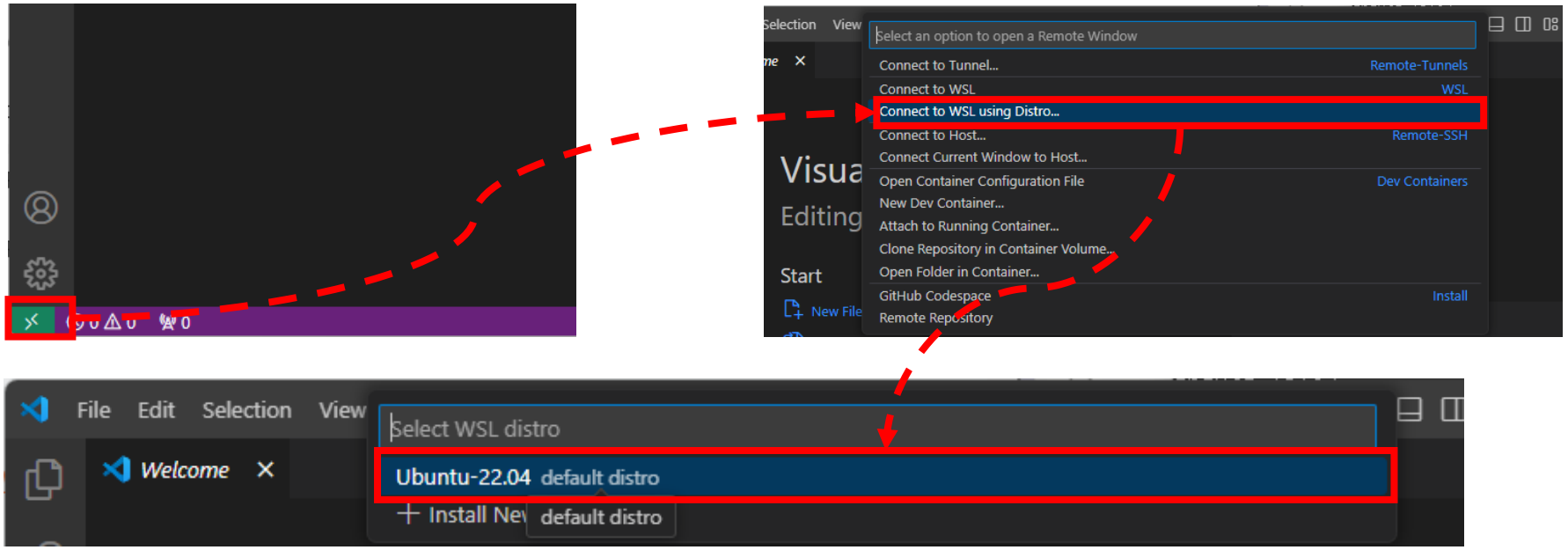


Download MobaXterm Home Edition, Portable



Linux 계정 등록,
Password 입력 시
입력된 문자가
없는 것처럼 보임

Preparing dev. env. (VS code)



Open folder → '/home/<user_name>'
Linux 위의 환경을 VS code로 사용가능

Prepare: Package requirements

Ctrl + `(어퍼스트로피) → Terminal 입력 (Shell)

Shell

```
$ sudo apt update
$ sudo apt install python3-pip -y
$ sudo apt install python3.10-dev python3.10-distutils -y
$ sudo apt install python3-tk libfuse2 graphviz libgraphviz-dev -y
$ sudo apt install python3-virtualenv -y
$ sudo apt-get install libgl1-mesa-dev
$ python --version # python version 표기
```

HAILO developer zone 가입

Developer zone의 리본 메뉴 → SW Download

ACCELERATORS → Dataflow Compiler

Python 버전에 맞춰 다운로드 → Ubuntu의 '/home/<user_name>'으로 복사

'hailo_dataflow_compiler-3.29.0-py3-none-linux_x86_64.whl'

Prepare: Create virtual env.

Pypi pkg: ultralytics, hailo_client는 numpy 버전 충돌

→ python 개발 환경 2개 필요

Shell

```
$ virtualenv ultralytics_env # 가상환경 생성
$ . ./ultralytics_env/bin/activate # 가상환경 실행
(ultralytics_env) $ pip install ultralytics jupyter
(ultralytics_env) $ deactivate # 가상환경 종료

$ virtualenv hailo_env
$ . ./hailo_env/bin/activate
(hailo_env) $ pip install hailo_dataflow_compiler_(…).whl
(hailo_env) $ hailo -h # Check install
(hailo_env) $ deactivate # 가상환경 종료
```

Prepare: Calibration data for PTQ

Dataflow Compiler의 Model Optimization 과정에서
Model Train에 사용한 Dataset의 input이 필요

→ ultralytics YOLO는 COCO2017로 사전훈련된 Model 제공

Shell

```
$ mkdir ./COCO2017

# 18 GB
$ wget http://images.cocodataset.org/zips/train2017.zip
    -O ./COCO2017/train2017.zip
$ unzip ./COCO2017/train2017.zip -d ./COCO2017/train2017
$ rm ./COCO2017/train2017.zip

# 800 MB
$ wget http://images.cocodataset.org/zips/val2017.zip
    -O ./COCO2017/val2017.zip
$ unzip ./COCO2017/val2017.zip -d ./COCO2017/val2017
$ rm ./COCO2017/val2017
```

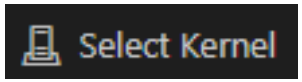
Exporting YOLO from ultralytics

Shell

```
$ mkdir ./tf_model # ultralytics로부터 파일을 받을 폴더
$ . ./ultralytics_env/bin/activate # 가상환경 실행
(ultralytics_env) $ python -m jupyter notebook
```

‘/home/<user_name>/<name_for_yolo_exporting>.ipynb’
파일 생성

ipynb 파일의 kernel로
방금 실행한 Jupyter notebook kernel의 url을 전달
url ex) http://127.0.0.1:8888/tree?token=6c5171e...



Exporting YOLO from ultralytics

Python (Jupyter cell)

```
import pathlib
import shutil # 디렉토리 조작, 파일 조작을 위한 라이브러리
import ultralytics
ultralytics.checks() # ultralytics package 정상동작 확인
```

Python (Jupyter cell)

```
model = ultralytics.YOLO('yolov8n.pt')
    # yolov8n 예시
Model.export(format = ‘saved_model’)
    # onnx 등의 방법이 있지만, opset 문제

# ./tf_model 디렉토리로 파일 이동
shutil.move(
    src = './yolov8n_saved_model',
    dst = './tf_model/yolov8n_saved_model'
)
```

Parsing model from Tensorlfow Lite

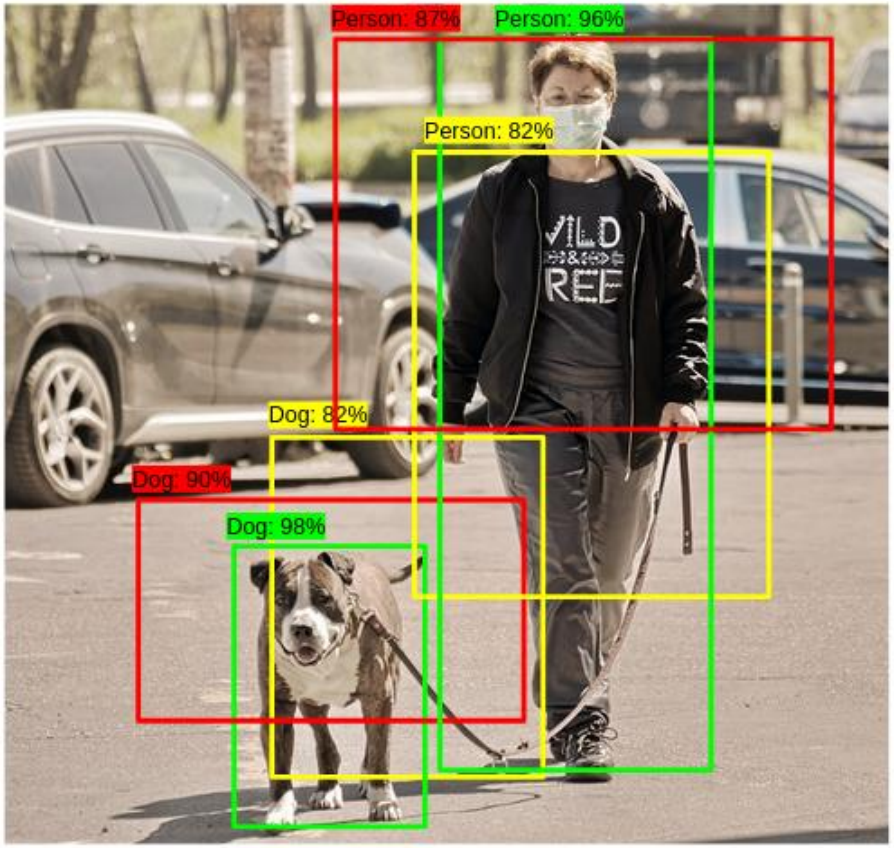
Shell

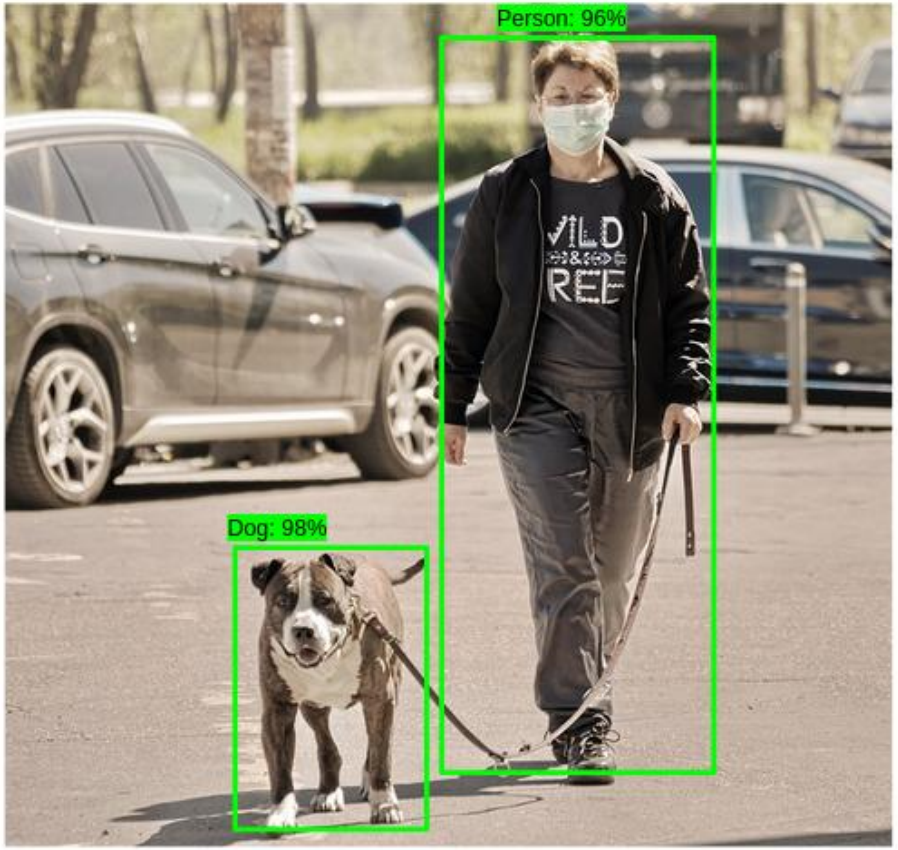
```
# Ctrl + C로 Jupyter kernel 종료
(ultralytics_env) $ deactivate

$ . ./hailo_env/bin/activate
(hailo_env) $ python -m jupyter notebook
```

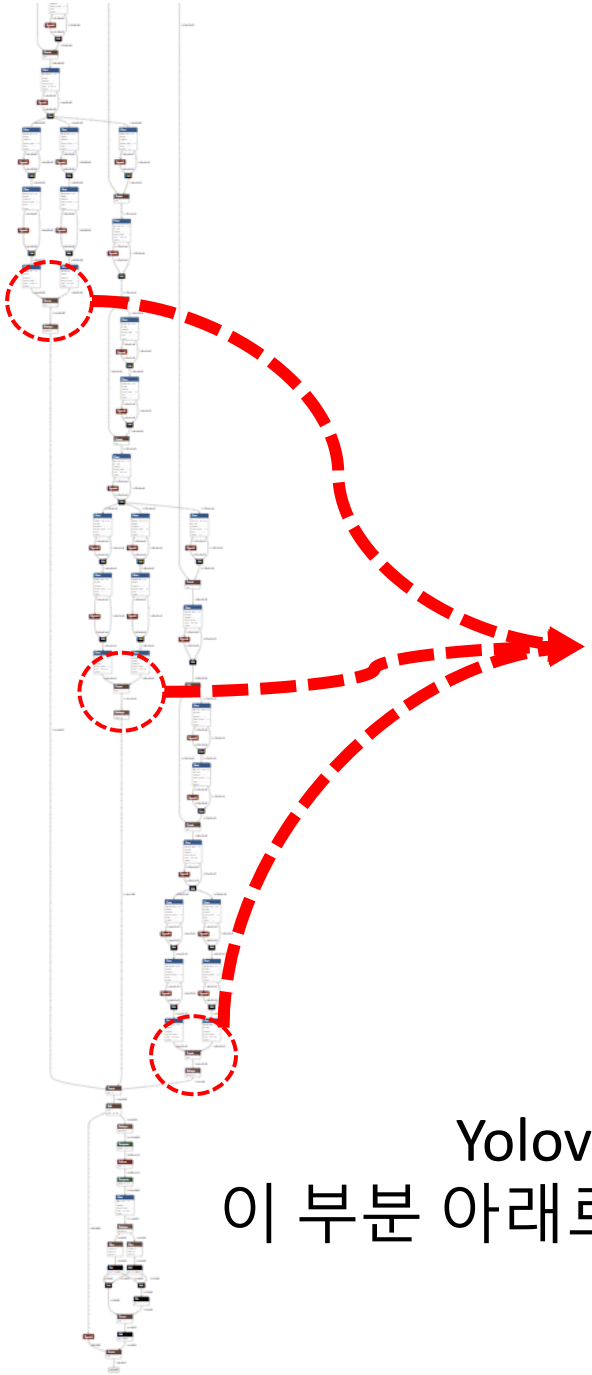
‘/home/<user_name>/<name_for_hailo_convert>.ipynb’
파일 생성 후 jupyte notebook kernel과 연결

Parsing: NMS in object detection

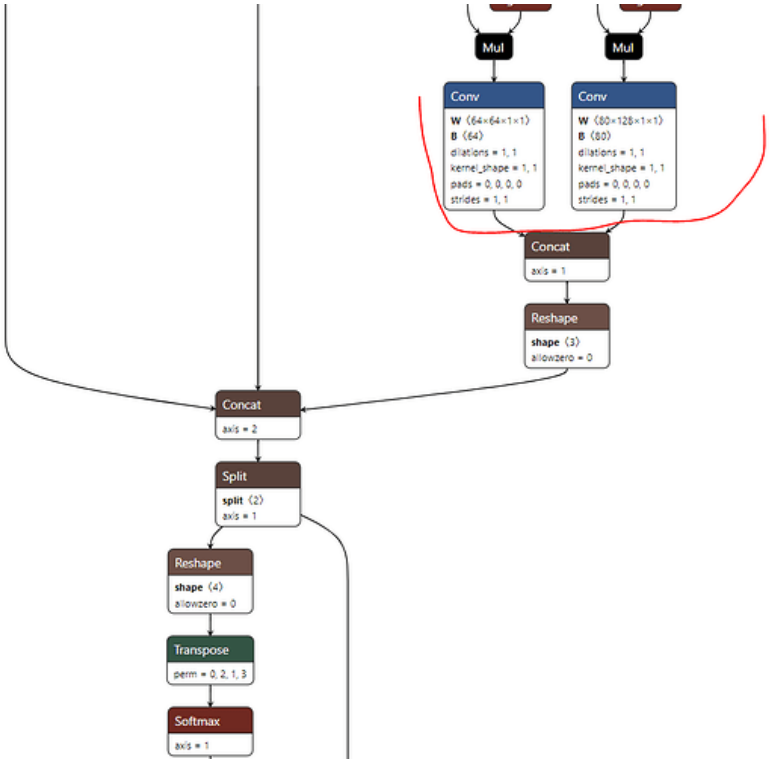




Object detection model은 중복 검출을 없애기 위해
Non Maximum Surpression 사용: 딥러닝의 post processing



NMS는
Hailo SoC에서 Layer로 지원하지 않음.



Yolov8n 기준
이 부분 아래로는 NMS 레이어

https://community.hailo.ai/t/parsing-yolo-models-with-the-hailo-dataflow-compiler-tool/2384

Parsing: Config for NMS layer

hailo_model_zoo / hailo_model_zoo / cfg / postprocess_config / yolov8n_nms_config.json

HailoModelZoo update-to-version-2.11.0

Code Blame 32 lines (32 loc) · 596 Bytes

```
{
        "nms_scores_th": 0.2,
        "nms_iou_th": 0.7,
        "image_dims": [
                640,
                640
        ],
        "max_proposals_per_class": 100,
        "classes": 80,
        "regression_length": 16,
        "background_removal": false,
        "bbox_decoders": [
                {
                        "name": "yolov8n/bbox_decoder41",
                        "stride": 8,
                        "reg_layer": "yolov8n/conv41",
                        "cls_layer": "yolov8n/conv42"
                },
                {
                        "name": "yolov8n/bbox_decoder52",
                        "stride": 16,
                        "reg_layer": "yolov8n/conv52",
                        "cls_layer": "yolov8n/conv53"
                },
                {
                        "name": "yolov8n/bbox_decoder62",
                        "stride": 32,
                        "reg_layer": "yolov8n/conv62",
                        "cls_layer": "yolov8n/conv63"
                }
        ]
}
```

Hailo github에서
NMS Node에 대한 자료 제공
(keras node name)

Parsing에서 제거한 NMS는
Model optimize 단계에서 붙일 수 있음

https://github.com/hailo-ai/hailo_model_zoo/tree/master/hailo_model_zoo/cfg/postprocess_config



yolov8n_float32.tflite

NMS Node

```
model_284/tf.math.add_1758/Add;model_284/tf.nn.convolution_1410/convolution
model_284/tf.math.add_1761/Add;model_284/tf.nn.convolution_1413/convolution
model_284/tf.math.add_1771/Add;model_284/tf.nn.convolution_1423/convolution
model_284/tf.math.add_1772/Add;model_284/tf.nn.convolution_1424/convolution
model_284/tf.math.add_1780/Add;model_284/tf.nn.convolution_1432/convolution
model_284/tf.math.add_1781/Add;model_284/tf.nn.convolution_1433/convolution
```

yolov8m_float32.tflite

NMS Node

```
model_316/tf.math.add_1942/Add;model_316/tf.nn.convolution_1556/convolution
model_316/tf.math.add_1943/Add;model_316/tf.nn.convolution_1557/convolution
model_316/tf.math.add_1956/Add;model_316/tf.nn.convolution_1570/convolution
model_316/tf.math.add_1955/Add;model_316/tf.nn.convolution_1569/convolution
model_316/tf.math.add_1966/Add;model_316/tf.nn.convolution_1580/convolution
model_316/tf.math.add_1967/Add;model_316/tf.nn.convolution_1581/convolution
```

https://netron.app/

Parsing model from Tensorlfow Lite

Python (Jupyter cell)

```
import pathlib

HAILO_MODEL_PROCESS_PATH = pathlib.Path('./hailo_model')
HAILO_MODEL_PROCESS_PATH.mkdir(exist_ok = True)
TFLITE_MODEL_PATH =
pathlib.Path('./tf_model/yolov8n_saved_model/yolov8n_float32.tflite')
```

Python (Jupyter cell)

```
import hailo_sdk_client
from hailo_sdk_client import ClientRunner

ACCELERATOR_CHIP_MODEL = 'hailo8l'
runner = hailo_sdk_client.ClientRunner(
    hw_arch = ACCELERATOR_CHIP_MODEL
)
```

Parsing model from Tensorlfow Lite

Python (Jupyter cell)

```
hn, npz = runner.translate_tf_model(
        model_path = TFLITE_MODEL_PATH,
        net_name = 'yolov8n',
        end_node_names = [ # NMS 절단을 위한 옵션
            'model_284/tf.math.add_1758/Add;model_284/tf.nn.convolution_1410/convolution',
            'model_284/tf.math.add_1761/Add;model_284/tf.nn.convolution_1413/convolution',
            'model_284/tf.math.add_1771/Add;model_284/tf.nn.convolution_1423/convolution',
            'model_284/tf.math.add_1772/Add;model_284/tf.nn.convolution_1424/convolution',
            'model_284/tf.math.add_1780/Add;model_284/tf.nn.convolution_1432/convolution',
            'model_284/tf.math.add_1781/Add;model_284/tf.nn.convolution_1433/convolution'
        ]
    )
runner.save_har(
    har_path = str(HAILO_MODEL_PROCESS_PATH/'yolov8n_parsed.har')
)
```

Python (Jupyter cell)

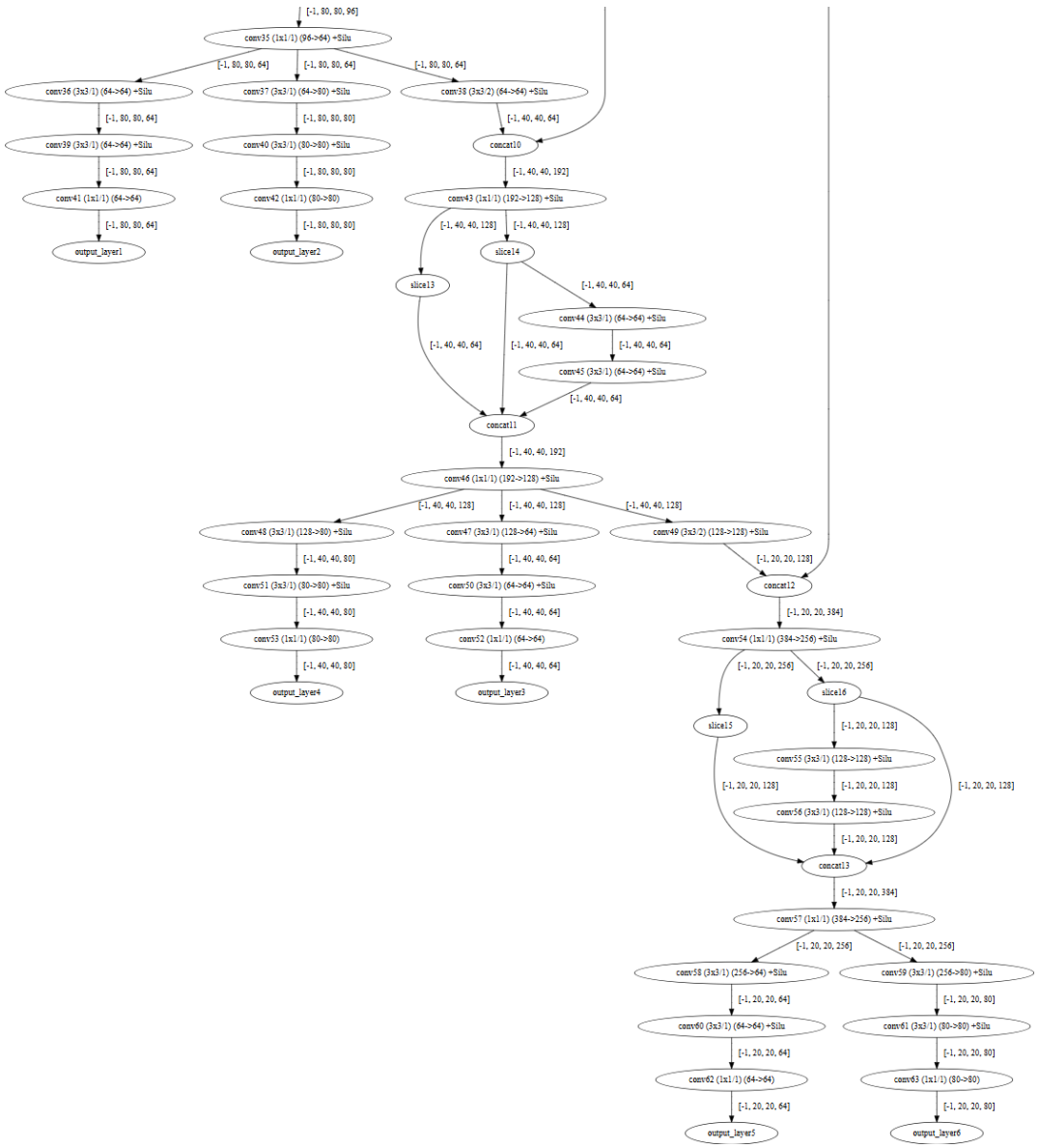
```
!hailo visualizer {str(HAILO_MODEL_PROCESS_PATH/'yolov8n_parsed.har')}
    --out-path {str(HAILO_MODEL_PROCESS_PATH/'yolov8n_parsed.svg')} --no-browser

import IPython
import IPython.display

IPython.display.SVG('./' + str(HAILO_MODEL_PROCESS_PATH/'yolov8n_parsed.svg'))
```

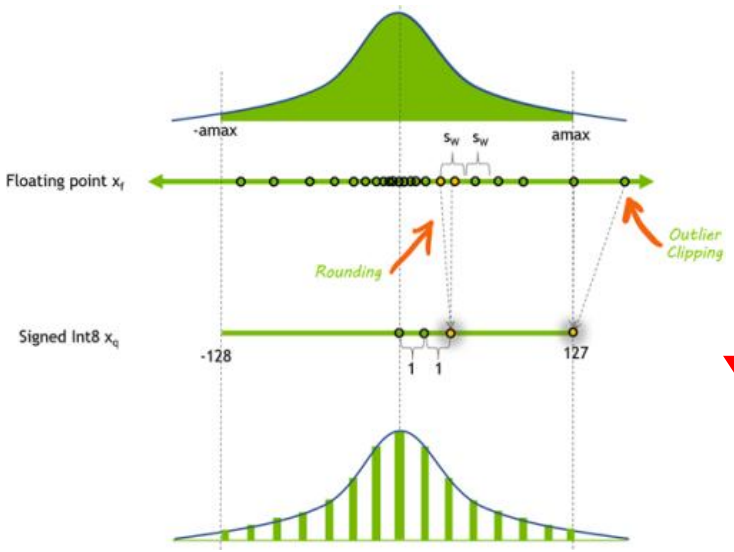
Result of parsing

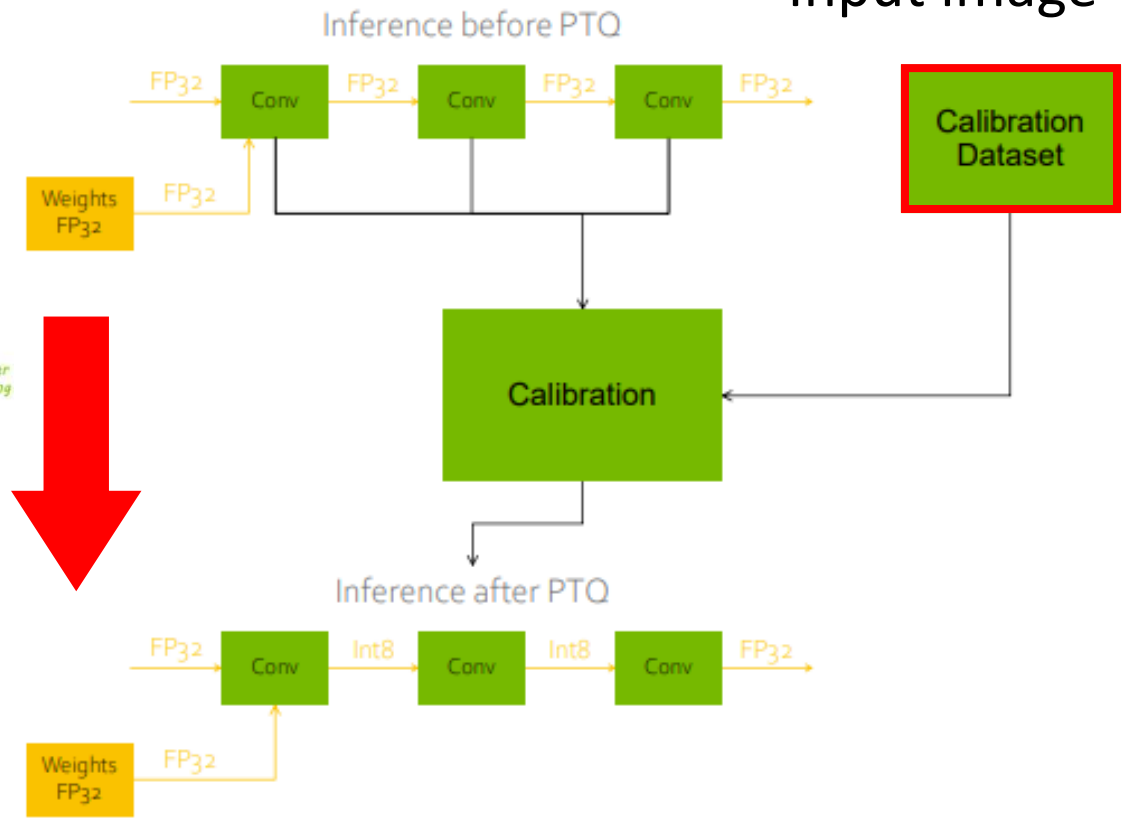




Model optimize: Quantization

Model train에 사용한
Input image





Model optimize
: Generate calibration dataset

Python (Jupyter cell)

```python
# Calibration dataset을 위한 Image list화
images_path = pathlib.Path("./COCO2017/train2017/train2017")
images_list = [img.name for img in images_path.iterdir() if img.suffix == ".jpg"]

import tensorflow as tf
from tensorflow.python.eager.context import eager_mode

# Image를 Calibration dataset으로 바꾸기 위한 함수
def preproc(image, output_height=640, output_width=640):
    """width and height for yolov8n input"""
    with eager_mode():
        h, w = image.shape[0], image.shape[1]
        scale = tf.cond( # resize scale target configuration
            pred = tf.less(h, w), # 짧은 방향으로 resize
            true_fn = lambda: output_height / h,
            false_fn = lambda: output_width / w
        )
        resized_image = tf.compat.v1.image.resize_bilinear(
            tf.expand_dims(image, 0), [int(h * scale), int(w * scale)]
        )
        cropped_image = tf.compat.v1.image.resize_with_crop_or_pad(
            resized_image, output_height, output_width
        )

        return tf.squeeze(cropped_image)
```

Model optimize
: Generate calibration dataset

Python (Jupyter cell)

```
import numpy as np
from PIL import Image
import random
import time

CALIBRATION_SET_SIZE = 1000 # minimum size: 1000
TARGET_MODEL_HEIGHT = 640
TARGET_MODEL_WIDTH = 640

# Calibration sample의 사이즈가 크므로 1000개 추출
random.seed(time.time())
sampled_images_list = random.sample(images_list, CALIBRATION_SET_SIZE)

calib_dataset = np.zeros((len(sampled_images_list), TARGET_MODEL_HEIGHT, TARGET_MODEL_WIDTH, 3))
for idx, img_name in enumerate(sorted(sampled_images_list)):
    current_img_path = images_path / img_name
    img = np.array(Image.open(current_img_path))
    try:
        img_preproc = preproc(img, output_height = TARGET_MODEL_HEIGHT, output_width =
TARGET_MODEL_WIDTH)
    except Exception as ex:
        # COCO2017의 흑백 이미지 제거
        print(f" {ex}")
        print(f"GRAYSCALE IMAGE SKIP:{img_name}")
        continue
    calib_dataset[idx, :, :, :] = img_preproc.numpy()
```

Model optimize(PTQ), compile

Python (Jupyter cell)

```
np.save("calib_set_random_COCO2017_train.npy", calib_dataset)

# float input에 대한 Normalization layer 파라메터
mean_per_channel = np.mean(calib_dataset, axis=(0, 1, 2))
std_per_channel = np.std(calib_dataset, axis=(0, 1, 2))

# optimize에서 추가되는 layer
# input normalization, NMS layer 추가
script_alls = [
    f"normalization1 = normalization({[round(value, 2) for value in mean_per_channel]},
{[round(value, 2) for value in std_per_channel]})\n",
    "nms_postprocess(meta_arch=yolov8, engine=cpu, nms_scores_th=0.2, nms_iou_th=0.4)\n"
]

runner.load_model_script("".join(script_alls))
runner.optimize(calib_dataset)
runner.save_har(
    har_path = str(HAILO_MODEL_PROCESS_PATH/'yolov8n_optimized.har')
)
```

Python (Jupyter cell)

```
hef = runner.compile()

file_name = str(HAILO_MODEL_PROCESS_PATH/f'yolov8n_{ACCELERATOR_CHIP_MODEL}.hef')
with open(file_name, "wb") as f:
    f.write(hef)
```

HailoRT on RPi5

Fix driver

Install dependencies and package

Inference the model

Fix: PCIe driver reinstall

Raspberry Pi 5가 SSH로 연결되어 있음을 가정

Developer zone의 리본 메뉴 → SW Download

ACCELERATORS → HailoRT

Python 버전에 알맞게 다운로드 → RPi의 '/home/<user_name>'으로 복사

'hailort-pcie-driver_4.19.0_all.deb'
'hailort_4.19.0_arm64.deb'
'hailort-4.19.0-cp311-cp311-linux_aarch64.whl'

Shell

```
$ sudo apt update
$ python --version # python version 표기

$ uname -r
<6.6.31+rpt-rpi-2712>
$ rm /lib/modules/6.6.31+rpt-rpi-2712
    /kernel/drivers/media/pci/hailo/hailo_pci.ko.xz
    # RPi OS에 사전 설치된 Driver 제거
```

Fix: PCIe driver reinstall

Shell

```
$ sudo apt install bison libelf-dev dkms
$ sudo dpkg --install hailort-pcie-driver_4.19.0_all.deb
$ sudo dpkg --install hailort_4.19.0_arm64.deb
$ sudo reboot now

# Kernel에서 제거 후 설정 변경하여 다시 load
$ sudo modprobe -r hailo_pci
$ sudo modprobe hailo_pci force_desc_page_size=4096

# Verify PCIe device is ready
$ lspci | grep Co-processor

# Verify driver is propely installed
$ lsmod | grep hailo_pci

# Verify firmware is loaded
$ dmesg | grep hailo
```

Install Python API

Shell

```
$ venv hailo_env --system-site-packages
    # picamera2 package는 가상환경에서 사용 불가
    # 시스템 패키지 사용 옵션 부여
$ . ./hailo_env/bin/activate
(hailo_env) $ pip install hailort-4.19.0-(…)-linux_aarch64.whl
(hailo_env) $ pip install opencv-python
```

Inference: picamera2 test

Python (Jupyter cell)

```
import cv2
from picamera2 import Picamera2

# picamera2 package load test
picam = Picamera2()
picam.configure(
    picam.create_preview_configuration(
        main = {
            "format": 'XRGB8888',
            'size': (1024, 768)
        }
    )
)
```

Inference: HailoRT inference test

Python (Jupyter cell)

```python
import numpy as np

from multiprocessing import Process
from hailo_platform import (
    HEF, VDevice, HailoStreamInterface,
    ConfigureParams, InputVStreamParams, OutputVStreamParams,
    InputVStreams, InferVStreams, OutputVStreams,
    FormatType
)

# Loading compiled HEFs to device:
hef_path = './yolov8n_float32.hef'
hef = HEF(hef_path)

# Configure network groups
configure_params = ConfigureParams.create_from_hef(
    hef = hef,
    interface = HailoStreamInterface.PCIe
)
```

Inference: HailoRT inference test

Python (Jupyter cell)

```
with VDevice() as target:
# why use 'with - as' keyword -> monopoly of resource
    network_groups = target.configure(hef, configure_params)
    network_group = network_groups[0]
    network_group_params = network_group.create_params()
    input_vstream_params = InputVStreamParams.make(
        network_group,
        format_type = FormatType.FLOAT32
    )
    output_vstream_params = OutputVStreamParams.make(
        network_group,
        format_type = FormatType.FLOAT32
    )
    # SHAPE FORMAT: HWC
    input_vstream_info = hef.get_input_stream_infos()
    height_input, width_input, channel_input = input_vstream_info[0].shape

    output_vstream_info = hef.get_output_stream_infos()
    # Generate random input dataset (dummy for inference test)
    low, high = 2, 20
    num_of_img = 1
    dataset = np.random.randint(
        low, high, (
            num_of_img, height_input, width_input, channel_input
        )
    ).astype(np.float32)
```

Inference: HailoRT inference test

Python (Jupyter cell)

```
with VDevice() as target:
    …
    …
    with InferVStreams(network_group, input_vstream_params, output_vstream_params) as
infer_pipeline:
        input_layer_info = input_vstream_info[0]
        input_data = { input_layer_info.name: dataset }
        with network_group.activate(network_group_params = network_group_params):
            infer_results = infer_pipeline.infer(input_data) # output of inference
```